



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE SOFTWARE

TEMA:

Practica Experimental 2

GRUPO:

CAJAS IBARRA IRVIN MARCELO
PANAMA MURILLO MOISES ANTONIO
TAIPE MORA ZAIDA MELISSA

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web – A – [5to Nivel]

<https://github.com/Zaida-tm18/PE-backend.git>

Introducción

Esta práctica compara dos tecnologías de backend muy usadas hoy en día: PHP 8.2 con PDO y Java 21 con Spring Boot 3. Ambas trabajan sobre la misma base de datos PostgreSQL 16 y corren juntas con Docker Compose, lo que permite ver de forma directa cómo cada una resuelve los mismos problemas.

El trabajo cubre tres temas principales. Primero, cómo funciona la programación del lado del servidor y la diferencia entre modelos como CGI, FastCGI y servidores embebidos. Segundo, el ecosistema Java para la web: Servlets, JSP y Spring Boot, con foco en seguridad y manejo de dependencias. Tercero, ASP.NET Core como punto de comparación, revisando su pipeline de middleware, las vistas Razor y los principios SOLID aplicados a los tres lenguajes.

En la parte práctica se construyó un sistema de login y un CRUD de mascotas en ambas tecnologías. PHP maneja la seguridad de forma manual: hash Argon2id, tokens CSRF, cabeceras HTTP y consultas preparadas con PDO. Spring Boot hace lo mismo pero de forma automática a través de Spring Security. Como las dos aplicaciones usan la misma base de datos, un dato creado en PHP aparece al instante en Spring Boot y viceversa.

Tema 1: Introducción a la Programación del Lado del Servidor

1.1 Programación del lado del servidor: conceptos generales

La programación del lado del servidor se apoya en una arquitectura de tres niveles, donde el nivel de lógica procesa las peticiones HTTP del cliente y se comunica con el nivel de datos [1]. Para que esto funcione, el servidor web necesita un modelo de integración con el intérprete del lenguaje, y existen varios enfoques contrastantes. El primero, CGI (Common Gateway Interface), crea un proceso nuevo del intérprete por cada solicitud HTTP, le pasa la petición mediante variables de entorno y recoge la salida por la salida estándar; es simple e independiente del lenguaje, pero el costo de crear y destruir un proceso en cada petición lo vuelve impracticable bajo carga alta [1]. FastCGI resuelve este problema manteniendo un grupo de procesos persistentes que se comunican con el servidor mediante un protocolo binario sobre sockets, en lugar de crear un proceso nuevo cada vez [1]; PHP-FPM es la implementación estándar actual de este modelo. Un tercer modelo es la integración como módulo embebido del servidor, como `mod_php` en Apache, donde el intérprete se carga dentro del propio proceso del servidor web; esto reduce la latencia, pero acopla el ciclo de vida del intérprete al del servidor y aumenta el consumo de memoria, por lo que la mayoría de despliegues mo-

ernos prefieren PHP-FPM sobre `mod_php`. Finalmente, los servidores embebidos –como el servidor de desarrollo de PHP, el contenedor *Servlet* embebido de Spring Boot o el módulo `http` nativo de Node.js– eliminan la necesidad de un servidor web externo, ya que la propia aplicación escucha directamente las conexiones TCP entrantes; este modelo es habitual en arquitecturas de microservicios y contenedores [1].

1.2 PERL para aplicaciones web

Históricamente, Perl fue uno de los lenguajes pioneros en el desarrollo web dinámico, utilizado extensamente a través de CGI gracias a su versatilidad para la manipulación de texto [2]. Sin embargo, la combinación de Perl con CGI se considera hoy una práctica obsoleta para infraestructuras públicas, ya que presenta vulnerabilidades de seguridad críticas y, como se vio en el subtema anterior, el propio modelo CGI es ineficiente bajo carga [1], [2]. A pesar de su declive comercial, Perl mantiene un nicho en aplicaciones científicas y bioinformáticas publicadas hace décadas, como SeqPHASE, que dependen de sus scripts para conversiones de formatos genómicos complejos [2]. La tendencia actual es reimplementar estas herramientas en lenguajes más modernos o en arquitecturas que eliminen la necesidad de mantener servidores CGI dedicados, evitando así los riesgos de seguridad asociados a este modelo de integración [2].

1.3 PHP: Hypertext Preprocessor

PHP es uno de los lenguajes de script más utilizados en el desarrollo de aplicaciones web dinámicas, integrando instrucciones directamente en archivos HTML que el servidor procesa para generar la respuesta enviada al cliente [3], [4]. PHP 8.x introdujo mejoras relevantes para este proyecto. El compilador JIT (*Just-In-Time*), incorporado en PHP 8.0, traduce *opcodes* a código máquina en tiempo de ejecución; su impacto es mayor en cargas de cómputo intensivo que en aplicaciones web típicas, donde el cuello de botella suele ser de entrada/salida, por lo que no reemplaza a OPcache [4]. Las *match expressions* (PHP 8.0) ofrecen una alternativa a `switch` con comparación estricta y sin *fallthrough*. Los *Fibers* (PHP 8.1) permiten pausar y reanudar funciones sin bloquear el hilo principal, base de las librerías asíncronas modernas. Las propiedades y clases *readonly* (PHP 8.1 y 8.2) favorecen la creación de objetos inmutables.

En seguridad de autenticación, PHP ofrece varios algoritmos de *hash* de contraseñas. Bcrypt utiliza un único factor de costo, pero no es *memory-hard*, lo que facilita ataques paralelizados

con GPU. Argon2, ganador de la *Password Hashing Competition* de 2015, ofrece las variantes Argon2i y Argon2id; esta última es recomendada por OWASP por combinar resistencia a ataques de canal lateral y a hardware paralelo mediante parámetros de memoria, tiempo y paralelismo (OWASP Foundation, 2024). En este proyecto se utiliza `PASSWORD_ARGON2ID` con `memory_cost=65536`, `time_cost=4` y `threads=1`, por encima del mínimo recomendado por OWASP (OWASP Foundation, 2024).

La gestión de sesiones es otro punto crítico de seguridad en PHP. La fijación de sesión (*session fixation*) ocurre cuando un atacante logra que la víctima use un identificador de sesión conocido antes de autenticarse; si la aplicación no genera uno nuevo, este puede reutilizarse para acceder a la sesión autenticada. El secuestro de sesión (*session hijacking*) consiste en robar un identificador válido mediante XSS o interceptación de tráfico. La mitigación estándar es invocar `session_regenerate_id(true)` tras cualquier cambio de privilegio, complementado con cookies `HttpOnly`, `Secure` y `SameSite` [5].

La inyección SQL ocurre cuando datos no confiables se concatenan directamente en una consulta, permitiendo alterar su estructura lógica. Por su impacto, forma parte de la categoría A03:2021 *Injection* del OWASP Top 10 (OWASP Foundation, 2021). PDO con sentencias preparadas neutraliza este vector separando la estructura de la consulta de los datos [6]. Dentro de PDO, `bindParam()` vincula la variable por referencia y `bindValue()` vincula el valor de forma inmediata; ambos previenen la inyección por igual, variando solo en temporización y flexibilidad de los argumentos (PHP Group, 2024). Herramientas de análisis como Yama, basadas en *opcodes*, permiten además detectar este tipo de vulnerabilidades antes del despliegue [7].

1.4 Comparativa tecnológica y criterios de selección

La selección de una tecnología de backend se basa en criterios como latencia, consumo de CPU y estabilidad bajo carga. PHP ha demostrado tiempos de reacción rápidos en aplicaciones impulsadas por datos, mientras que Node.js destaca en escenarios de tiempo real gracias a su modelo de entrada/salida no bloqueante [8]. Dentro del ecosistema PHP, frameworks ligeros como CodeIgniter muestran mayor eficiencia en el uso de recursos frente a opciones más pesadas como CakePHP o Yii, especialmente con miles de usuarios concurrentes [9]. Otro criterio decisivo es la seguridad integrada por defecto: frameworks como Laravel son preferidos para sistemas críticos por sus módulos nativos de autenticación y protección contra inyección SQL y CSRF [6]. La madurez de la plataforma y el soporte de la comunidad son factores adicionales que influyen en la viabilidad a largo plazo de un proyecto [10].

Tema 2: Desarrollo Web con Java y JSP

2.1 Java para el desarrollo web: plataforma y herramientas

Java es un lenguaje orientado a objetos robusto y escalable, ampliamente utilizado en aplicaciones empresariales [11]. Dentro de este ecosistema destacan dos patrones de acceso a datos. El patrón Active Record integra en una misma clase los datos del modelo y la lógica de persistencia, lo que simplifica aplicaciones pequeñas, pero acopla el modelo a la base de datos [12]. El patrón Repository, en cambio, separa la persistencia en una clase dedicada y deja el modelo libre de detalles de acceso a datos, favoreciendo el principio de Inversión de Dependencias de SOLID y facilitando pruebas, sustitución de la fuente de datos o uso de *mocks* [12]. La clase `MascotaRepository` implementada en este proyecto con PHP y PDO es un ejemplo directo de este segundo enfoque.

2.2 Java Servlets

Los Java Servlets son componentes que se ejecutan en el servidor y procesan peticiones HTTP de forma dinámica dentro de un contenedor como Apache Tomcat [11], [13]. En arquitecturas modernas, las solicitudes atraviesan una tubería de filtros o *middleware* antes de llegar al controlador final. El orden de ejecución es crítico: primero deben preservarse el contexto y el manejo de excepciones, y después aplicarse autenticación y autorización [1]. Este mecanismo permite centralizar controles de seguridad, auditoría y transformación de datos, y constituye la base sobre la que más adelante opera Spring Security [1]. Aunque hoy se prefieren frameworks de mayor nivel, los servlets siguen siendo la base técnica de JSP y Spring.

2.3 JavaServer Pages (JSP)

JavaServer Pages (JSP) es una tecnología de servidor que permite insertar código Java dentro de archivos basados en etiquetas para generar interfaces dinámicas; internamente, cada JSP se traduce en un servlet ejecutado por la JVM [1], [13]. JSP resulta útil como capa de vista en arquitecturas MVC, al separar la representación visual de la lógica de negocio [13]. Aunque es una tecnología clásica, sigue utilizándose en portales con visualización de datos en tiempo real e integración con AJAX. Como en cualquier tecnología de presentación, el desarrollador debe sanitizar correctamente el contenido para prevenir vulnerabilidades como XSS [11].

2.4 Introducción a Spring Boot para desarrollo web

Spring Boot simplificó el desarrollo web en Java mediante configuración basada en anotaciones e inyección de dependencias, usando componentes como `@RestController`, `@Service` y `@Repository` [12]. Su contenedor de DI administra distintos ciclos de vida para los componentes, siendo *Singleton* el alcance predeterminado, junto con otros como *Prototype* o los asociados a petición y sesión.

La seguridad se gestiona mediante Spring Security, que organiza la autenticación y la autorización en una *filter chain* expuesta a través de `FilterChainProxy` y `SecurityFilterChain`. Cada filtro cumple una función específica y se ejecuta en un orden definido, de modo que ninguna solicitud llega al controlador sin pasar por controles de contexto de seguridad, autenticación y autorización [14]. Esta arquitectura también permite insertar filtros personalizados, por ejemplo para validar JWT.

Los principios SOLID se reflejan claramente en esta arquitectura. El principio de **Responsabilidad Única** se observa cuando `MascotaRepository` se encarga solo de la persistencia; el principio **Abierto/Cerrado** permite extender reglas de negocio sin modificar el código existente; la **Sustitución de Liskov** exige que distintas implementaciones del repositorio sean intercambiables; la **Segregación de Interfaces** favorece contratos pequeños y específicos; y la **Inversión de Dependencias** permite que los controladores dependan de abstracciones mientras el contenedor decide qué implementación concreta inyectar en tiempo de ejecución [12].

Tema 3: Desarrollo Web con ASP.NET

3.1 La plataforma .NET y ASP.NET

La plataforma .NET es el entorno de ejecución sobre el que corre el código escrito en C#, F# o Visual Basic, mantenido por Microsoft desde inicios de los años 2000. ASP.NET nació como la capa de esa plataforma dedicada a construir sitios y servicios web, y durante su primera década estuvo atada casi exclusivamente a Windows e IIS mediante el modelo Web Forms, que generaba HTML a partir de controles de servidor con estado persistente. Ese modelo resultó problemático para aplicaciones modernas por su acoplamiento al servidor y su dificultad para escribir pruebas automatizadas [15].

La ruptura llegó con ASP.NET Core, una reescritura completa del runtime que eliminó la dependencia de Windows y permitió ejecutar aplicaciones en Linux y macOS sin cambios. Este

salto arquitectónico tuvo consecuencias medibles en adopción: la popularidad de ASP.NET tuvo un pico alto hace más de una década, bajó considerablemente con la mala fama de Web Forms, y en años recientes volvió a niveles altos impulsada precisamente por ASP.NET Core [16]. En comparaciones directas de rendimiento contra Spring Boot, Express.js, Laravel y Django bajo arquitectura REST API, ASP.NET Core obtuvo los tiempos de respuesta más cortos y produjo imágenes Docker de menor tamaño [17], aunque ese margen de rendimiento no lo convierte automáticamente en la mejor opción para cualquier proyecto, ya que frameworks como Django o Laravel compensan con código más compacto y ecosistemas más accesibles para equipos pequeños.

La elección de .NET como plataforma implica también una decisión sobre el ciclo de soporte: las versiones LTS reciben soporte durante tres años, mientras que las versiones estándar reciben dieciocho meses. Esta política afecta directamente la planificación del PFC: una aplicación construida sobre una versión no LTS puede quedar sin parches de seguridad antes de que el proyecto madure. Para el contexto universitario y el mercado ecuatoriano, donde el sector bancario y el sector público muestran la mayor demanda de perfiles .NET, la estabilidad del soporte de Microsoft representa una ventaja competitiva frente a frameworks con ciclos de versión menos predecibles [16][17].

3.2 ASP.NET Core: estructura de un proyecto web y pipeline de middleware

Un proyecto en ASP.NET Core arranca con menos archivos que la versión clásica del framework. Su punto de entrada es `Program.cs`, donde se configura el servidor, se registran los servicios en el contenedor de inyección de dependencias y se define la secuencia de middleware que procesa cada solicitud antes de llegar al código de negocio [15]. La carpeta `wwwroot` agrupa los recursos estáticos que el navegador puede solicitar directamente —como hojas de estilo, imágenes y JavaScript—, separándolos del código del servidor y evitando la exposición accidental de archivos sensibles.

El pipeline de middleware funciona como una cadena de delegados: cada componente recibe la solicitud, puede modificarla o detenerla, y luego cede el control al siguiente. El orden de registro en `Program.cs` es determinista y afecta tanto la seguridad como el funcionamiento de la aplicación [15]. Un orden recomendado es el siguiente:

```
app.UseExceptionHandler("/Error");  
app.UseHsts();  
app.UseHttpsRedirection();
```

```

app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

```

Este orden es importante por razones concretas. Si `UseAuthorization()` se ejecuta antes de `UseAuthentication()`, las rutas protegidas no reciben la identidad del usuario. Si `UseStaticFiles()` se coloca después de la autenticación, archivos públicos como CSS o imágenes pasarían innecesariamente por controles de seguridad, afectando el rendimiento [15].

ASP.NET Core incorpora inyección de dependencias de forma nativa. Su contenedor define tres *lifetimes* principales [12][14]:

- **Transient** (`AddTransient<I,T>()`): crea una instancia nueva cada vez que el servicio es solicitado. Es adecuado para componentes sin estado y de bajo costo.
- **Scoped** (`AddScoped<I,T>()`): crea una instancia por solicitud HTTP. Es el lifetime apropiado para repositorios y `DbContext`, ya que mantiene consistencia durante una misma petición.
- **Singleton** (`AddSingleton<I,T>()`): crea una única instancia para toda la vida de la aplicación, por lo que solo es seguro en servicios completamente *thread-safe*.

Inyectar un servicio *Scoped* dentro de un *Singleton* provoca el problema conocido como *Captive Dependency*, porque el objeto de larga vida retiene una dependencia ligada a una petición concreta. En el PFC, el repositorio se registra e inyecta de la siguiente forma [12]:

```

// Program.cs
builder.Services.AddScoped<IMascotaRepository, JdbcMascotaRepository>();

// MascotasController.cs
public class MascotasController : Controller {
    private readonly IMascotaRepository _repo;
    public MascotasController(IMascotaRepository repo) {
        _repo = repo;
    }
}

```

3.3 Razor Pages y Razor Views

Razor es el motor de plantillas de ASP.NET Core que permite combinar HTML con código C# dentro del mismo archivo usando el símbolo `@` para delimitar expresiones del lado del servidor [15]. Razor Pages organiza cada página alrededor de un archivo `.cshtml` y su clase *code-behind* (`.cshtml.cs`), mientras que Razor Views se emplea en el patrón MVC, donde el controlador selecciona la vista y le envía el modelo correspondiente.

```
@model IEnumerable<Mascota>
<table>
@foreach (var m in Model) {
    <tr>
        <td>@m.Nombre</td>
        <td>@m.Especie</td>
        <td>@m.Edad</td>
    </tr>
}
</table>
```

Cuando Razor evalúa expresiones como `@m.Nombre`, escapa automáticamente caracteres especiales como `<`, `>` o comillas, lo que reduce el riesgo de XSS sin que el desarrollador tenga que aplicar el escape manualmente en cada vista [18].

No obstante, cualquier motor de plantillas puede ser vulnerable a *Server-Side Template Injection* (SSTI) si se construyen plantillas a partir de datos externos sin sanitización adecuada. En escenarios graves, este tipo de vulnerabilidad puede derivar en ejecución remota de código [19]. En Razor, la operación que exige mayor cuidado es `Html.Raw()`, ya que desactiva el escape automático y solo debería usarse con contenido interno previamente auditado [19][15].

La elección entre Razor Pages y MVC con Razor Views depende del tipo de aplicación. Razor Pages suele ser más apropiado para aplicaciones centradas en páginas con lógica local por pantalla, mientras que MVC resulta preferible cuando varias vistas comparten lógica de controlador o cuando la misma capa de negocio debe servir vistas HTML y endpoints JSON [16]. En el PFC se optó por MVC para mantener consistencia arquitectónica con la estructura usada también en Spring Boot [12].

3.4 Formularios, validación, seguridad y principios SOLID en ASP.NET Core

ASP.NET Core integra un sistema de validación de modelos basado en atributos de anotación de datos. Las propiedades decoradas con `[Required]`, `[StringLength]`, `[Range]` o `[EmailAddress]` son verificadas automáticamente antes de que el controlador procese la solicitud; el resultado se almacena en `ModelState` y puede mostrarse en la vista sin lógica adicional [15]:

```
public class MascotaViewModel {
    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(100, MinimumLength = 2)]
    public string Nombre { get; set; }

    [Required]
    public string Especie { get; set; }

    [Range(0, 30, ErrorMessage = "Edad entre 0 y 30 años")]
    public int Edad { get; set; }
}
```

La seguridad en formularios abarca tres frentes. Primero, la protección contra XSS: Razor escapa el contenido por defecto, aunque los formularios HTML siguen siendo un punto de entrada frecuente para inyectar código malicioso [18]. Segundo, la protección contra CSRF: ASP.NET Core genera tokens antifalsificación mediante `@Html.AntiForgeryToken()` y los valida en envíos POST con `[ValidateAntiForgeryToken]` [5]. Tercero, la protección contra inyección SQL: Entity Framework Core con consultas parametrizadas o JDBC con prepared statements evitan la interpolación directa de datos del usuario en el SQL [20].

Desde la perspectiva de seguridad integrada, ASP.NET Core centraliza la configuración en el pipeline de middleware (`Program.cs`), mientras que Spring Boot lo hace en la `SecurityFilterChain`. En Spring Boot, esta cadena de filtros se ejecuta antes del `DispatcherServlet` y gestiona el contexto de seguridad, la validación CSRF, la autenticación y la autorización [14][21]. La línea `.anyRequest().authenticated()` protege todas las rutas de forma declarativa, a diferencia del `AuthMiddleware::requireAuth()` en PHP, que debe invocarse manualmente en cada ruta [14].

La aplicación de los principios SOLID al backend del PFC permite identificar buenas prácticas y oportunidades de mejora [22]. El **Single Responsibility Principle** se cumple cuando

`MascotaRepository` gestiona el acceso a datos y el controlador el flujo HTTP; se viola cuando el controlador mezcla validación, persistencia y formateo de respuesta. El **Open/Closed Principle** se aplica al definir `IMascotaRepository` como interfaz, permitiendo añadir implementaciones sin modificar el controlador. El **Liskov Substitution Principle** garantiza que `PdoMascotaRepository` y `JdbcMascotaRepository` sean intercambiables al cumplir el mismo contrato [22]. El **Interface Segregation Principle** se refleja en la separación entre `IMascotaRepository` e `IUsuarioRepository`. Finalmente, el **Dependency Inversion Principle** se materializa cuando el controlador recibe la interfaz del repositorio por constructor, y el contenedor de DI de ASP.NET Core o Spring Boot resuelve la implementación en runtime [12][22].

Una violación frecuente de SOLID en proyectos académicos es el uso de métodos estáticos para el acceso a datos (`MascotaRepository::findAll()`), ya que impide la inyección de dependencias, dificulta las pruebas y acopla la lógica de negocio al motor de base de datos. La corrección consiste en extraer una interfaz, registrar la implementación en el contenedor de DI e inyectarla por constructor [22][12].

La comparación entre el contenedor IoC de Spring Boot y el sistema de DI de ASP.NET Core muestra dos enfoques sobre el mismo principio de inversión de control. ASP.NET Core registra manualmente los servicios en `Program.cs` con lifetimes como `AddScoped`, `AddTransient` o `AddSingleton`, mientras que Spring Boot usa anotaciones y escaneo de componentes como `@Service`, `@Repository` y `@Component` [15][12] [21]. Ambos enfoques implementan el principio DIP de SOLID, aunque Spring Boot reduce el código de configuración a costa de hacer menos visible qué está registrado.

En el contexto de la autenticación, Spring Security incorpora componentes especializados: `UserDetailsService` define cómo cargar un usuario desde la base de datos, mientras que `AuthenticationManager` delega la verificación en uno o más `AuthenticationProvider`, que comparan las credenciales con las almacenadas usando `BCryptPasswordEncoder` [14][21]. En el PFC, la implementación de `UserDetailsService` conecta Spring Security con `JdbcMascotaRepository` para verificar la existencia del usuario y la coincidencia de su contraseña hashada, cerrando el vínculo entre la filter chain de seguridad y el patrón Repository documentado en ADR-001 [21][12].

Resultados de la Implementación (Proyecto)

Paso 1 – Configuración del entorno de desarrollo

El entorno se levantó con Docker Compose, orquestando cuatro contenedores: **php-app** (PHP 8.2 + Apache, puerto 8080), **spring-app** (Java 21 + Spring Boot 3, puerto 8090), **postgres_db** (PostgreSQL 16, puerto 5432) y **adminer** (puerto 8081) como cliente web de administración.

La base de datos veterinaria fue creada en PostgreSQL a partir del script `php-app/sql/schema.sql`, montado como script de inicialización del contenedor de base de datos. En ella se generaron las tablas **usuarios** y **mascotas**, necesarias para la autenticación y el módulo CRUD.



Figura 1: Acceso a Adminer mostrando el motor de base de datos PostgreSQL, el servidor postgres-db y la base veterinaria.

Antes de ejecutar las pruebas visuales del sistema, se realizó el análisis estático del código PHP con PHPStan en nivel 5, sobre el directorio `src/` del módulo PHP, mediante el siguiente comando: `docker-compose exec php-app composer install docker-compose exec php-app vendor/bin/phpstan analyse src/ -level=5`

Paso 2 – Autenticación segura en PHP

En la versión PHP se implementó un registro de usuarios en el que las contraseñas se almacenan mediante la función `password_hash()` utilizando el algoritmo **Argon2id**, configurado con un costo de memoria de 65536 KB, un factor de tiempo de 4 y 1 hilo de procesamiento. Para el inicio de sesión se emplea la función `password_verify()`, evitando la comparación de contraseñas en texto plano. Además, tras autenticar al usuario, se regenera el identificador



Figura 2: Esquema public de la base de datos veterinaria en PostgreSQL, con las tablas mascotas y usuarios

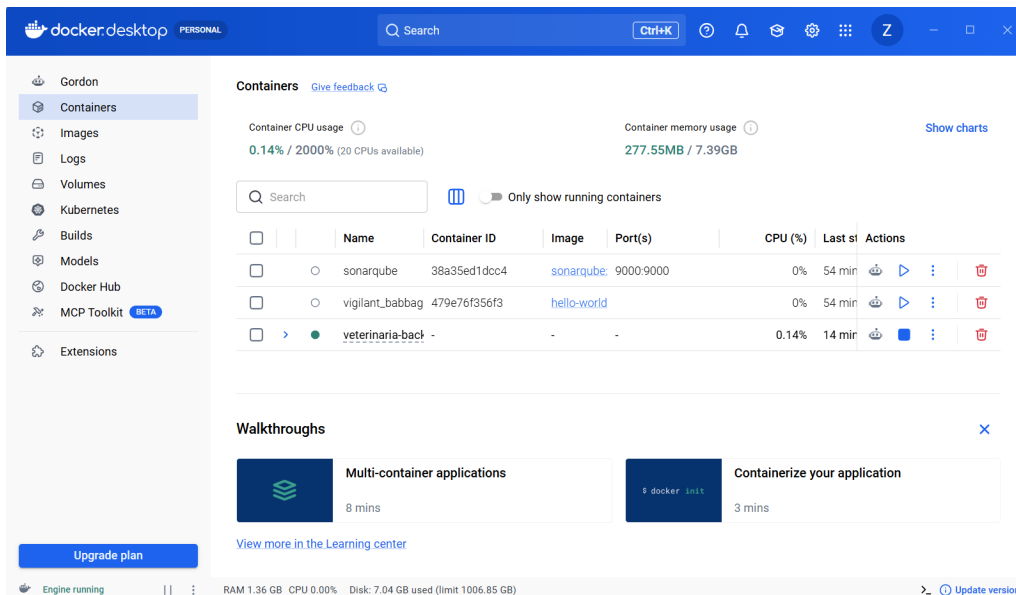


Figura 3: Contenedores Docker activos del proyecto (veterinaria-backend), visibles en Docker Desktop

de sesión con el fin de reducir el riesgo de fijación de sesión (*session fixation*).

Cada formulario incluye un token CSRF generado mediante `random_bytes(32)` y validado en el servidor antes de procesar la solicitud, utilizando una comparación en tiempo constante



Figura 4: Pantalla de bienvenida tras un inicio de sesión exitoso en el sistema PHP (puerto 8080)



Figura 5: Pantalla de inicio de sesión del sistema PHP



Figura 6: Pantalla de registro de un nuevo usuario en el sistema PHP

con la función `hash_equals()`. Esta medida impide que una petición enviada desde un sitio externo ejecute acciones sin la autorización del usuario autenticado.

Listing 1: Generación y verificación del token CSRF en PHP

```
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

function csrf_verify(): void {
```

```

    if (!hash_equals($_SESSION['csrf_token'],
                      $_POST['csrf_token'] ?? '')) {
        http_response_code(403);
        exit('CSRF_check_failed');
    }
}

```

Adicionalmente, la aplicación PHP envía cabeceras HTTP de seguridad en cada respuesta, entre ellas **X-Frame-Options**, **X-Content-Type-Options**, **Content-Security-Policy** y **Referrer-Policy**. Estas cabeceras ayudan a mitigar ataques de tipo *clickjacking*, *MIME sniffing* y la exposición innecesaria de información a través del encabezado **Referer**.

Listing 2: Cabeceras HTTP de seguridad configuradas en PHP

```

header('X-Frame-Options:_DENY');
header('X-Content-Type-Options:_nosniff');
header("Content-Security-Policy:_default-src_'self'");
header('Referrer-Policy:_strict-origin-when-cross-origin');

```

Criterio de verificación: El análisis con PHPStan (nivel 5) no reporta errores en la lógica de autenticación.

Paso 3 – CRUD con patrón Repository y PDO

El módulo de mascotas fue desarrollado utilizando el patrón *Repository* junto con PDO. Esta estructura permite desacoplar el acceso a datos de la lógica de negocio mediante una interfaz, **MascotaRepositoryInterface**, y su implementación concreta, **PdoMascotaRepository**. Las operaciones principales del módulo CRUD —crear, listar, visualizar detalle, editar y eliminar— se ejecutan mediante *prepared statements*, evitando la concatenación directa de los datos proporcionados por el usuario dentro de las sentencias SQL y reduciendo así el riesgo de inyección SQL.

Listing 3: Operaciones **create**, **update** y **delete** del repositorio de mascotas en PDO

```

public function create(array $d): void {
    $s = $this->pdo->prepare(
        'INSERT INTO mascotas (nombre, especie, edad, dueño_id)
        VALUES (?, ?, ?, ?)'
    );
}

```

```

        $s->execute ([ $d [ 'nombre' ], $d [ 'especie' ],
                      $d [ 'edad' ], $d [ 'duenio_id' ] ] );
    }

    public function update (int $id, array $d): void {
        $s = $this->pdo->prepare (
            'UPDATE_mascotas
            _____SET_nombre=_,_especie=_,_edad=_,
            _____WHERE_id=_'
        );
        $s->execute ([ $d [ 'nombre' ], $d [ 'especie' ],
                      $d [ 'edad' ], $id ] );
    }

    public function delete (int $id): void {
        $s = $this->pdo->prepare (
            'DELETE_FROM_mascotas_WHERE_id=_'
        );
        $s->execute ([ $id ] );
    }
}

```



Figura 7: Listado de mascotas registradas en la versión PHP (estado inicial, sin registros)

Criterio de verificación: Las 5 operaciones del CRUD funcionan correctamente. Se verifica que ninguna *query* usa concatenación de strings con datos del usuario.



Figura 8: Listado de mascotas tras registrar dos mascotas nuevas en PHP, confirmando el funcionamiento del CRUD

Paso 4 – Segunda tecnología: ASP.NET Core 8 / Java Spring Boot 3

La segunda implementación se desarrolló con *Spring Boot 3* y *JdbcTemplate* sobre *Java 21*. Esta versión conserva la misma funcionalidad principal de la aplicación desarrollada en PHP: autenticación de usuarios y un módulo CRUD de mascotas, ambos operando sobre la misma base de datos PostgreSQL compartida. En materia de seguridad, *Spring Security* proporciona de forma predeterminada mecanismos como la protección CSRF en formularios, el control de acceso a rutas y la gestión de sesiones.

Listing 4: Configuración declarativa de seguridad en Spring Security

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/", "/login", "/register", "/css/**"
            ).permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
```

```

        .loginPage("/login")
        .defaultSuccessUrl("/mascotas", true)
    )
    .headers(headers -> headers
        .frameOptions(
            HeadersConfigurer.FrameOptionsConfig::deny
        )
    );
    return http.build();
}

```



Figura 9: Pantalla de bienvenida tras un inicio de sesión exitoso en el sistema Spring Boot (puerto 8090)

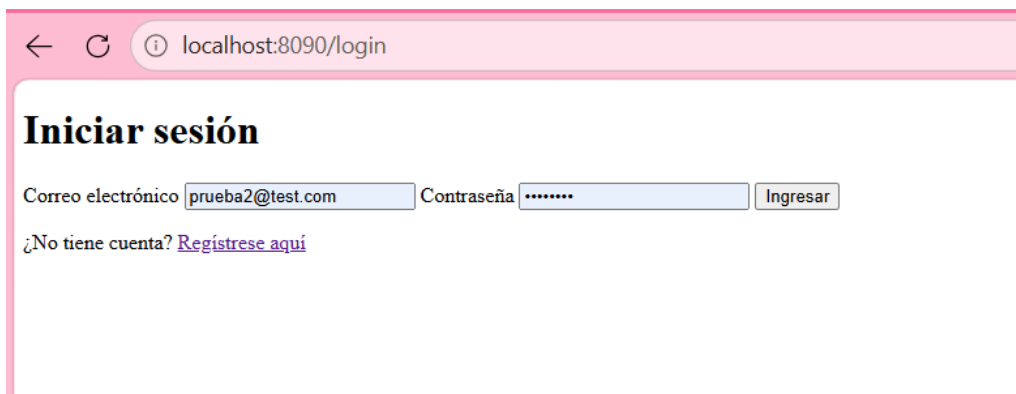


Figura 10: Pantalla de inicio de sesión del sistema Spring Boot

Ambas aplicaciones leen y escriben sobre la misma base de datos PostgreSQL: una mascota creada desde la aplicación PHP es visible de inmediato en el listado de Spring Boot, y viceversa. Esto demuestra que el mismo módulo CRUD funciona de forma equivalente sin importar la tecnología utilizada.

← ↻ ⓘ localhost:8090/register

Crear cuenta

Nombre completo Correo electrónico Contraseña Confirmar contraseña

¿Ya tiene cuenta? [Inicie sesión](#)

Figura 11: Pantalla de registro de un nuevo usuario en el sistema Spring Boot

← ↻ ⓘ localhost:8090/mascotas

Mascotas registradas

[« Volver al panel](#) | [+ Registrar nueva mascota](#)

Nombre	Especie	Raza	Propietario	Teléfono	Acciones
No hay mascotas registradas todavía.					

Figura 12: Listado de mascotas registradas en la versión Spring Boot (estado inicial)

← ↻ ⓘ localhost:8090/mascotas/crear

Registrar mascota

Nombre de la mascota Especie Raza (opcional) Fecha de nacimiento (opcional) Nombre del propietario Teléfono del propietario (opcional)

[« Volver al listado](#)

Figura 13: Formulario de creación de una nueva mascota en Spring Boot

← ↻ ⓘ localhost:8090/mascotas/editar/3

Editar mascota

Nombre de la mascota Especie Raza (opcional) Fecha de nacimiento (opcional) Nombre del propietario Teléfono del propietario (opcional)

[« Volver al listado](#)

Figura 14: Formulario de edición de una mascota existente en Spring Boot



Figura 15: Confirmación de operación exitosa tras registrar la mascota en Spring Boot, mostrando el listado actualizado

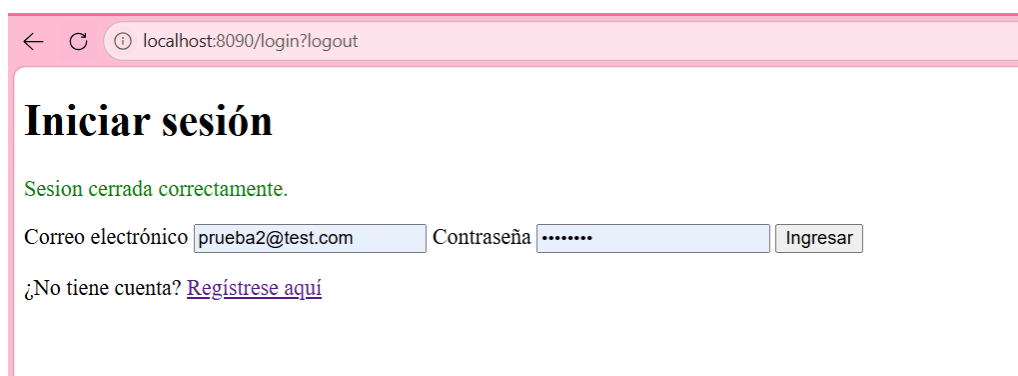


Figura 16: Cierre de sesión en el sistema Spring Boot; al intentar volver a /mascotas, Spring Security redirige automáticamente al login

Tabla 1: Comparativa técnica: PHP 8.2 + PDO vs. ASP.NET Core 8 / Spring Boot 3

Criterio	PHP 8.2 + PDO	ASP.NET Core 8 / Spring Boot 3
Líneas de código (auth)	≈150–200 líneas sin framework; ≈80 con Laravel Breeze	ASP.NET: ≈60 con Identity; Spring: ≈90 con Spring Security
Curva de aprendizaje	Baja: sintaxis directa, se puede empezar sin framework [4]	Media-alta: requiere comprender inyección de dependencias y pipeline de middleware [12]
Seguridad por defecto	Manual: el desarrollador implementa CSRF, escape XSS y cabeceras; Laravel lo automatiza [7]	Alta: escape automático en Razor/Thymeleaf, tokens CSRF integrados y HSTS en una línea [17]
Rendimiento (req/s)	≈12 000–18 000 req/s con OP-cache habilitado [8]	ASP.NET Core: ≈45 000–80 000; Spring Boot: ≈20 000–35 000 req/s [17]
Ecosistema de paquetes	Packagist: >350 000 paquetes; Composer como gestor estándar [4]	NuGet (>300 000) y Maven Central (>500 000 artefactos) con gestión madura [23]
Hosting en Ecuador	Ampliamente disponible: la mayoría de proveedores locales incluyen PHP/MySQL sin costo extra	Menos común en hosting compartido; requiere VPS o Docker (mayor costo operativo)
Comunidad	Muy grande: millones de repositorios y foros activos [4]	Grande y corporativa: soporte oficial de Microsoft (ASP.NET) y VMware (Spring) [16]
Integración con Cloud	Soportado en AWS, GCP y Azure; imágenes Docker oficiales disponibles	Primera clase en Azure (ASP.NET) y AWS Elastic Beanstalk (Spring); CI/CD nativos [14]
Facilidad de pruebas	PHPUnit maduro; PHPStan para análisis estático [7]	xUnit/.NET Test Explorer (ASP.NET) y JUnit 5 con Mockito (Spring); integración nativa en IDEs [12]
Salario promedio dev	Ecuador: USD 800–1 400/mes (perfil junior-mid PHP) según portales de empleo locales 2024	Ecuador: USD 1 200–2 200/mes (perfil .NET/Java mid-senior); mayor demanda en banca y sector público

Paso 5 – Análisis de seguridad y documentación de decisiones

Conforme al Paso 5.2 de la guía, se documentaron dos Architecture Decision Records (ADR) en el repositorio del proyecto, bajo docs/adr/, referenciados desde el README.md del repositorio en la sección "Decisiones de arquitectura".

ADR-001: Selección de tecnología backend

Estado: Aceptado.

Decisión: se implementó el backend con PHP 8.2 (obligatorio) como primera tecnología y Java 21 + Spring Boot 3.2 como segunda tecnología, ambas usando PDO/JDBC con prepared statements y el patrón Repository (interfaz + implementación concreta). Para el acceso a datos en Spring Boot se eligió JDBC puro, en lugar de Spring Data JPA, de modo que la comparación con PHP/PDO fuera directa: ambas implementaciones controlan explícitamente la apertura de conexión, la preparación de la sentencia y el mapeo fila-a-objeto, sin un ORM intermedio que oculte esos pasos.

Alternativas consideradas: se descartó ASP.NET Core porque el equipo cuenta con más experiencia previa en Java, y se descartó Spring Data JPA para mantener la comparación de verbosidad de acceso a datos justa frente a PDO. Consecuencias: el código de los repositorios en ambas tecnologías es estructuralmente paralelo (mismas cinco operaciones CRUD, mismo uso explícito de prepared statements), lo que facilita justificar cada criterio de la tabla comparativa con evidencia de código real. Como contraparte, usar JDBC puro implica más código repetitivo de mapeo manual que el que tendría una aplicación Spring Boot con JPA.

ADR-002: Estrategia de base de datos

Estado: Aceptado.

Decisión: se utiliza una sola base de datos PostgreSQL 16 compartida entre las dos aplicaciones backend, definida una única vez en php-app/sql/schema.sql y montada como script de inicialización del contenedor postgres-db. Ni PHP ni Spring Boot generan o migran el esquema por sí mismos, precisamente para que ambas tecnologías lean y escriban sobre la misma estructura sin divergencias. Las contraseñas se almacenan siempre como hash (Argon2id en PHP, BCrypt en Spring Boot) y el acceso a datos se realiza exclusivamente mediante prepared statements.

Alternativas consideradas: se descartó usar una base de datos independiente por tecnología, porque introduciría el riesgo de que ambas bases de datos divergieran en estructura o contenido, invalidando la comparación pedida en el Paso 4 de la guía. Consecuencias: cualquier

dato creado desde la aplicación PHP es visible de inmediato desde la aplicación Spring Boot y viceversa, lo que permite demostrar que ambos backends son intercambiables sobre el mismo dominio de datos. Como contraparte, un cambio futuro en el esquema de una tabla debe replicarse manualmente en el código de mapeo de ambos repositorios.

5.7 Repositorio y control de versiones

El código fuente del proyecto fue gestionado mediante *Git* y organizado dentro del repositorio `veterinaria-backend`, estructurado en los directorios `php-app/`, `spring-app/` y `docs/`. Esta organización permitió separar los componentes correspondientes a cada tecnología utilizada, así como centralizar la documentación y los archivos de apoyo del proyecto.

Como parte de la evidencia solicitada, se debe incluir el historial de *commits* del repositorio, en el que se refleje la participación de todos los integrantes del equipo. Este requisito está establecido tanto en el Paso 5 de la guía de la práctica como en la directriz (6) de la rúbrica de evaluación.

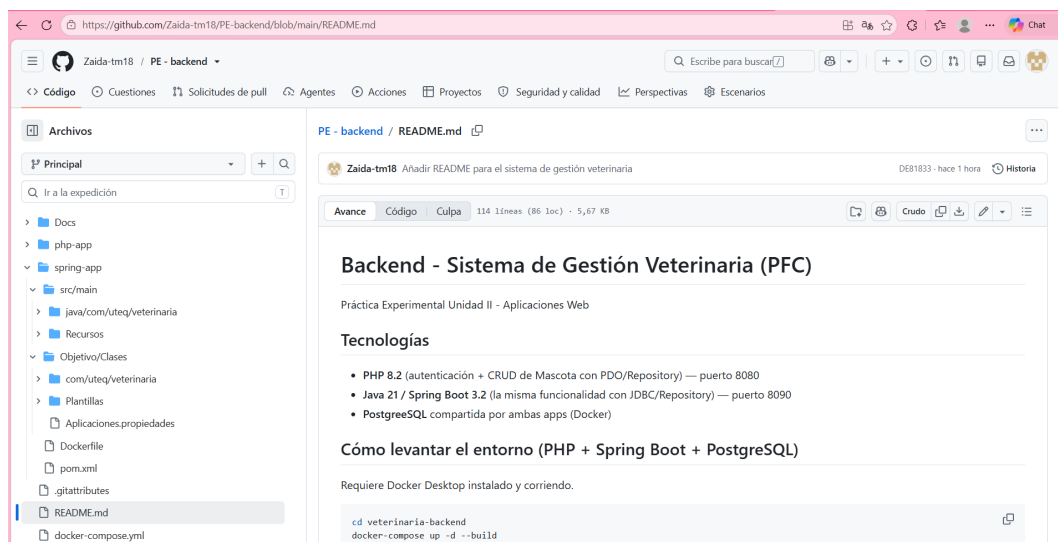


Figura 17: Estructura general del repositorio `veterinaria-backend`.

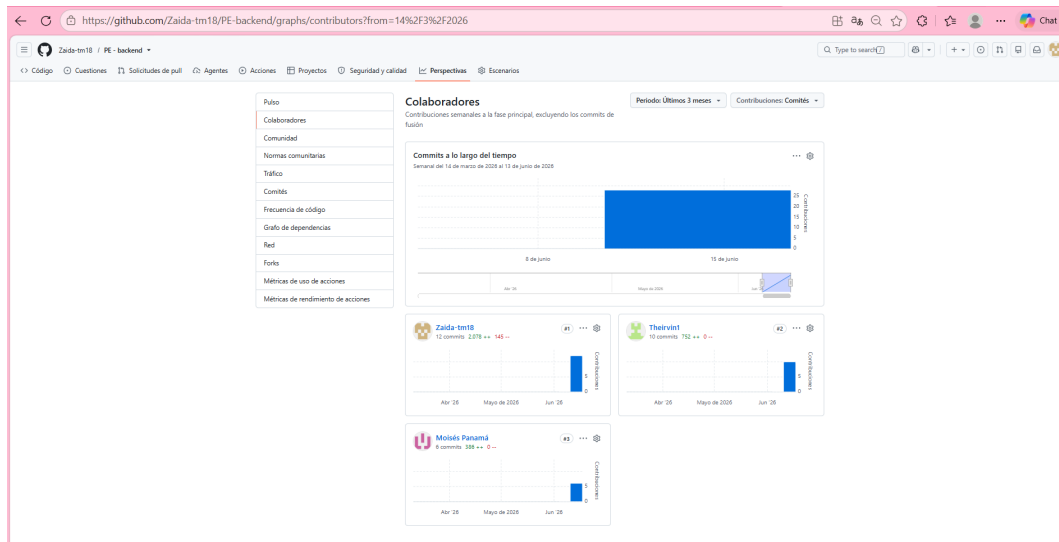


Figura 18: Commits del repositorio **veterinaria-backend**.

Conclusión

La práctica dejó claro que elegir una tecnología no es solo una cuestión de preferencia, sino que tiene consecuencias reales en seguridad, cantidad de código y facilidad de mantenimiento.

En PHP, cada protección —CSRF, escape de datos, cabeceras de seguridad, regeneración de sesión— tuvo que escribirse a mano. En Spring Boot, esas mismas protecciones vienen activadas por defecto dentro de la cadena de filtros de Spring Security. Esto no significa que PHP sea inseguro, sino que exige más atención del desarrollador en cada punto de la aplicación.

Usar JDBC puro en lugar de Spring Data JPA fue una buena decisión para esta práctica: obligó a escribir el acceso a datos de la misma forma en ambas tecnologías, haciendo la comparación más justa. El patrón Repository, aplicado en los dos backends, demostró que es posible separar la lógica de negocio del acceso a la base de datos de manera clara y ordenada.

En cuanto a qué tecnología usar, PHP sigue siendo la opción más práctica para proyectos pequeños o académicos en Ecuador: es más fácil de aprender, el hosting es barato y hay mucha comunidad local. Spring Boot tiene mejor rendimiento y mayor demanda en empresas grandes y bancos, pero requiere más configuración y conocimiento previo. Lo ideal es conocer los dos.

Referencias

- [1] V. Gervasi, E. Börger y A. Cisternino, “Modeling Web Applications Infrastructure with ASMs,” *Science of Computer Programming*, vol. 94, págs. 69-92, 2014. [10.1016/j.scico.2014.02.025](https://doi.org/10.1016/j.scico.2014.02.025).
- [2] Y. Spöri y J.-F. Flot, “Haxe as a Swiss Knife for Bioinformatic Applications: the SeqPHASE Case Story,” *Briefings in Bioinformatics*, vol. 25, n.º 5, 2024. [10.1093/bib/bbae367](https://doi.org/10.1093/bib/bbae367).
- [3] R. Rasha, M. M. Khan, M. Masud y M. A. Al-Zain, “Investigain: A Productive Asset Management Web Application,” *Computer Systems Science and Engineering*, vol. 38, n.º 2, págs. 151-164, 2021. [10.32604/CSSE.2021.015314](https://doi.org/10.32604/CSSE.2021.015314).
- [4] A. Rio y F. Brito e Abreu, “PHP Code Smells in Web Apps: Evolution, Survival and Anomalies,” *Journal of Systems and Software*, vol. 200, 2023. [10.1016/j.jss.2023.111644](https://doi.org/10.1016/j.jss.2023.111644).
- [5] X. Likaj, S. Khodayari y G. Pellegrino, “Where We Stand (or Fall): An Analysis of CSRF Defenses in Web Frameworks,” en *ACM International Conference Proceeding Series*, 2021, págs. 370-385. [10.1145/3471621.3471846](https://doi.org/10.1145/3471621.3471846).
- [6] J. Adamu, R. Hamzah y M. M. Rosli, “Security Issues and Framework of Electronic Medical Record: A Review,” *Bulletin of Electrical Engineering and Informatics*, vol. 9, n.º 2, págs. 565-572, 2020. [10.11591/eei.v9i2.2064](https://doi.org/10.11591/eei.v9i2.2064).
- [7] J. Zhao, K. Zhu, L. Yu, H. Huang e Y. Lu, “Yama: Precise Opcode-Based Data Flow Analysis for Detecting PHP Applications Vulnerabilities,” *IEEE Transactions on Information Forensics and Security*, vol. 20, págs. 7748-7763, 2025. [10.1109/TIFS.2025.3592537](https://doi.org/10.1109/TIFS.2025.3592537).
- [8] Q. Odeniran, H. Wimmer y C. M. Rebman, “Node.js or PHP? Determining the Better Website Server Backend Scripting Language,” *Issues in Information Systems*, vol. 24, n.º 1, págs. 328-341, 2023. [10.48009/1_iis_2023_128](https://doi.org/10.48009/1_iis_2023_128).
- [9] K. Rak y M. Dzieńkowski, “Comparative Analysis of Web Development Frameworks in PHP: CodeIgniter, CakePHP and Yii,” *Informatyka, Automatyka, Pomiary w Gospodarce i Ochronie Srodowiska*, vol. 16, n.º 1, págs. 155-161, 2026. [10.35784/iapgos.8358](https://doi.org/10.35784/iapgos.8358).
- [10] P. Vuorimaa, M. Laine, E. Litvinova y D. Shestakov, “Leveraging Declarative Languages in Web Application Development,” *World Wide Web*, vol. 19, n.º 4, págs. 519-543, 2016. [10.1007/s11280-015-0339-z](https://doi.org/10.1007/s11280-015-0339-z).
- [11] K. Takahashi, “A Comparative Case Study of Two Pedagogical Approaches in Web Development Education: From a Traditional Java Environment to a Modern Ruby on Rails Ecosystem,” *Science, Engineering and Technology*, vol. 5, n.º 2, págs. 20-28, 2025. [10.54327/set2025/v5.i2.305](https://doi.org/10.54327/set2025/v5.i2.305).

- [12] B. Y. Alkazemi y M. K. Nour, “XBoot: A RAPID and Instructional Low-Code Generator for Spring Boot Applications,” *Applied Sciences*, vol. 15, n.º 19, 2025. [10.3390/app151910621](https://doi.org/10.3390/app151910621).
- [13] J. Kim, I. Yeo, H. Kim, A. Sohn, Y. Kim e Y. Kim, “Web Portal for Analytical Validation of MRM-MS Assay Abided with Integrative Multinational Guidelines,” *Scientific Reports*, vol. 10, n.º 1, 2020. [10.1038/s41598-020-67731-x](https://doi.org/10.1038/s41598-020-67731-x).
- [14] C. L. Aldea y R. Bocu, “Authentication Challenges and Solutions in Microservice Architectures,” *Applied Sciences*, vol. 15, n.º 22, 2025. [10.3390/app152212088](https://doi.org/10.3390/app152212088).
- [15] D. P. Kouru, “Design and implementation of secure web applications using ASP.NET Core and .NET framework,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 18, n.º 3, págs. 341-350, 2026. [10.30574/wjaets.2026.18.3.0160](https://doi.org/10.30574/wjaets.2026.18.3.0160).
- [16] J. Swacha y A. Kulpa, “Evolution of Popularity and Multiaspectual Comparison of Widely Used Web Development Frameworks,” *Electronics*, vol. 12, n.º 17, pág. 3563, 2023. [10.3390/electronics12173563](https://doi.org/10.3390/electronics12173563).
- [17] M. Szewczyk y M. Skublewska-Paszkowska, “Performance comparison of development frameworks in selected environments in REST API architecture,” *Journal of Computer Sciences Institute*, vol. 35, págs. 121-128, 2025. [10.35784/jcsi.7041](https://doi.org/10.35784/jcsi.7041).
- [18] T. Zhang, H. Huang, Y. Lu, K. Zhu y J. Zhao, “State-Sensitive Black-Box Web Application Scanning for Cross-Site Scripting Vulnerability Detection,” *Applied Sciences*, vol. 13, n.º 16, pág. 9212, 2023. [10.3390/app13169212](https://doi.org/10.3390/app13169212).
- [19] L. Pisu, D. Maiorca y G. Giacinto, “An Assessment of the Overlooked Dangers of Template Engines,” *ACM Transactions on the Web*, 2025. [10.1145/3799796](https://doi.org/10.1145/3799796).
- [20] K. Sambhus e Y. Liu, “Automating SQL Injection and Cross-Site Scripting Vulnerability Remediation in Code,” *Software*, vol. 3, n.º 1, págs. 28-46, 2024. [10.3390/software3010002](https://doi.org/10.3390/software3010002).
- [21] B. Atul, R. More, S. G. Kewadkar, Y. Namdev y M. Deshpande, “Authorization Security and Identity Management in Web Application Development Using Java and Spring Boot Security,” *International Journal on Advanced Computer Theory and Engineering*, vol. 15, n.º 2S, págs. 247-254, 2026. [10.65521/ijacte.v15i2S.3002](https://doi.org/10.65521/ijacte.v15i2S.3002).
- [22] N. C. Ramachandrappa, “SOLID Design Principles in Software Engineering,” *International Journal of Computer Trends and Technology*, vol. 72, n.º 9, págs. 18-23, 2024. [10.14445/22312803/IJCTT-V72I9P104](https://doi.org/10.14445/22312803/IJCTT-V72I9P104).

- [23] J. K. Ponte-Isminio, G. J. Huerta-Macedo, P. Castañeda, S. Wong-Durand, D. Mauricio y A. Oñate-Andino, “Web Application Based on Artificial Intelligence for the Control of Healthy Habits in People with Unbalanced Diet in Lima,” *International Journal of Online and Biomedical Engineering*, vol. 21, n.º 12, págs. 121-141, 2025. [10.3991/ijoe.v21i12.55599](https://doi.org/10.3991/ijoe.v21i12.55599).

Anexo C – Preguntas de Análisis

C.1 [Análisis] Seguridad por defecto: PHP 8.2 vs. Spring Boot 3

Al implementar el mismo módulo de autenticación en ambas tecnologías, Spring Boot 3 resultó más seguro por defecto. En PHP 8.2 sin framework, varias protecciones debieron implementarse manualmente: tokens CSRF con `bin2hex(random_bytes(32))` y validación mediante `hash_equals()`, cabeceras HTTP de seguridad (`X-Frame-Options`, `Content-Security-Policy`, `Referrer-Policy`), escape de salida con `htmlspecialchars()` y regeneración de sesión tras el login con `session_regenerate_id(true)` [5].

Spring Boot 3 con Spring Security incorporó automáticamente tokens CSRF en formularios, escape de salida en Thymeleaf, cabeceras de seguridad HTTP, gestión de sesión e invalidación automática, además de protección declarativa de rutas con `.anyRequest().authenticated()` [14]. La diferencia es arquitectónica: PHP delega la seguridad al desarrollador en cada punto de la aplicación, mientras que Spring Security centraliza estas decisiones en la *filter chain*, reduciendo el riesgo de omitir controles [15]. Esta observación coincide con ADR-001, donde se reconoce que PHP requiere implementar manualmente controles que Spring Boot ofrece por defecto.

C.2 [Evaluación] Recomendación tecnológica para el PFC y proyectos futuros en Ecuador

Considerando el contexto del mercado tecnológico de Los Ríos y Ecuador, la recomendación es **PHP 8.2 para el PFC**, con la proyección de incorporar Spring Boot en proyectos de mayor escala o con clientes del sector bancario y público.

Esta recomendación se sustenta en tres factores. Primero, la disponibilidad de hosting: muchos proveedores locales incluyen PHP y MySQL sin costo adicional, mientras que Spring Boot suele requerir VPS o contenedores Docker, elevando el costo operativo. Segundo, la disponibilidad de desarrolladores: PHP cuenta con una base más amplia de profesionales

para mantenimiento local [4]. Tercero, la curva de aprendizaje: PHP permite producir funcionalidad con menor complejidad inicial que Spring Boot, que exige comprender inyección de dependencias y arquitectura por capas [4][12].

No obstante, para proyectos con alta concurrencia o clientes institucionales, Spring Boot es la opción superior: reporta entre 20 000 y 35 000 req/s frente a 12 000–18 000 de PHP con OPcache [17][8]. Además, la ausencia de *connection pooling* nativo en PHP/PDO, documentada en ADR-002, representa una limitación real bajo carga, mientras que Spring Boot lo resuelve con HikariCP [8].

C.3 [Síntesis] Resultados de PHPStan nivel 5 y refactorizaciones aplicadas

PHPStan nivel 5 reportó errores y advertencias al analizar el directorio `src/`. Las categorías más frecuentes fueron: tipos de retorno no declarados en métodos de los repositorios, acceso a índices de arreglos sin verificación previa (`Offset 'id' might not exist on array`) y parámetros con tipo `mixed` implícito en funciones auxiliares de `csrf.php`.

Las refactorizaciones aplicadas incluyeron la declaración explícita de tipos de retorno en `PdoMascotaRepository`, la sustitución de accesos directos a `$_POST` por variables validadas con `filter_input()`, y la adición de verificaciones `isset()` antes de acceder a índices de resultados de consulta [7].

Para detectar estos problemas antes del análisis estático, el proceso de desarrollo debería integrar PHPStan como paso obligatorio del pipeline CI/CD en `.github/workflows/phpstan.yml`, ejecutándose en cada *pull request*. De forma complementaria, el uso de *type hints* estrictos y `declare(strict_types=1)` ayudaría a detectar muchos de estos errores desde el IDE [7].

C.4 [Evaluación] Integración con el frontend de PE-U1 y diagrama de la primera petición autenticada

La integración del backend con el frontend de PE-U1 se realizaría mediante una API REST que exponga endpoints JSON consumidos por el cliente JavaScript. El backend debe habilitar CORS restringido al origen del frontend y utilizar cookies de sesión con atributos `HttpOnly` y `SameSite=Strict` [5]. El flujo de la primera petición autenticada sería el siguiente:

1. El usuario ingresa sus credenciales en el formulario de login del frontend.
2. El frontend envía `POST /auth/login` al backend.

3. El backend valida el token CSRF, recupera el usuario por email y verifica la contraseña con `password_verify()` en PHP o `BCryptPasswordEncoder.matches()` en Spring Boot.
4. Si las credenciales son correctas, PHP ejecuta `session_regenerate_id(true)` e inicia la sesión; Spring Boot actualiza el `SecurityContext` y genera la cookie de sesión [5][14].
5. El backend responde 200 OK con los datos del usuario autenticado.
6. El frontend redirige al dashboard protegido e incluye la cookie de sesión en las siguientes peticiones.
7. El frontend solicita GET `/mascotas` con la cookie incluida.
8. El backend verifica la autenticación mediante `AuthMiddleware::requireAuth()` en PHP o `AuthorizationFilter` en Spring Boot.
9. El backend consulta la base de datos mediante el repositorio correspondiente (`PdoMascotaRepository` o `JdbcMascotaRepository`) y responde con el listado en JSON.
10. El frontend renderiza los datos en la interfaz del usuario.

Este flujo garantiza que los datos protegidos no sean accesibles sin autenticación previa y que la sesión se mantenga segura frente a fijación y robo mediante las contramedidas descritas en la sección 3.4, en coherencia con la estrategia documentada en ADR-002 [5][15].